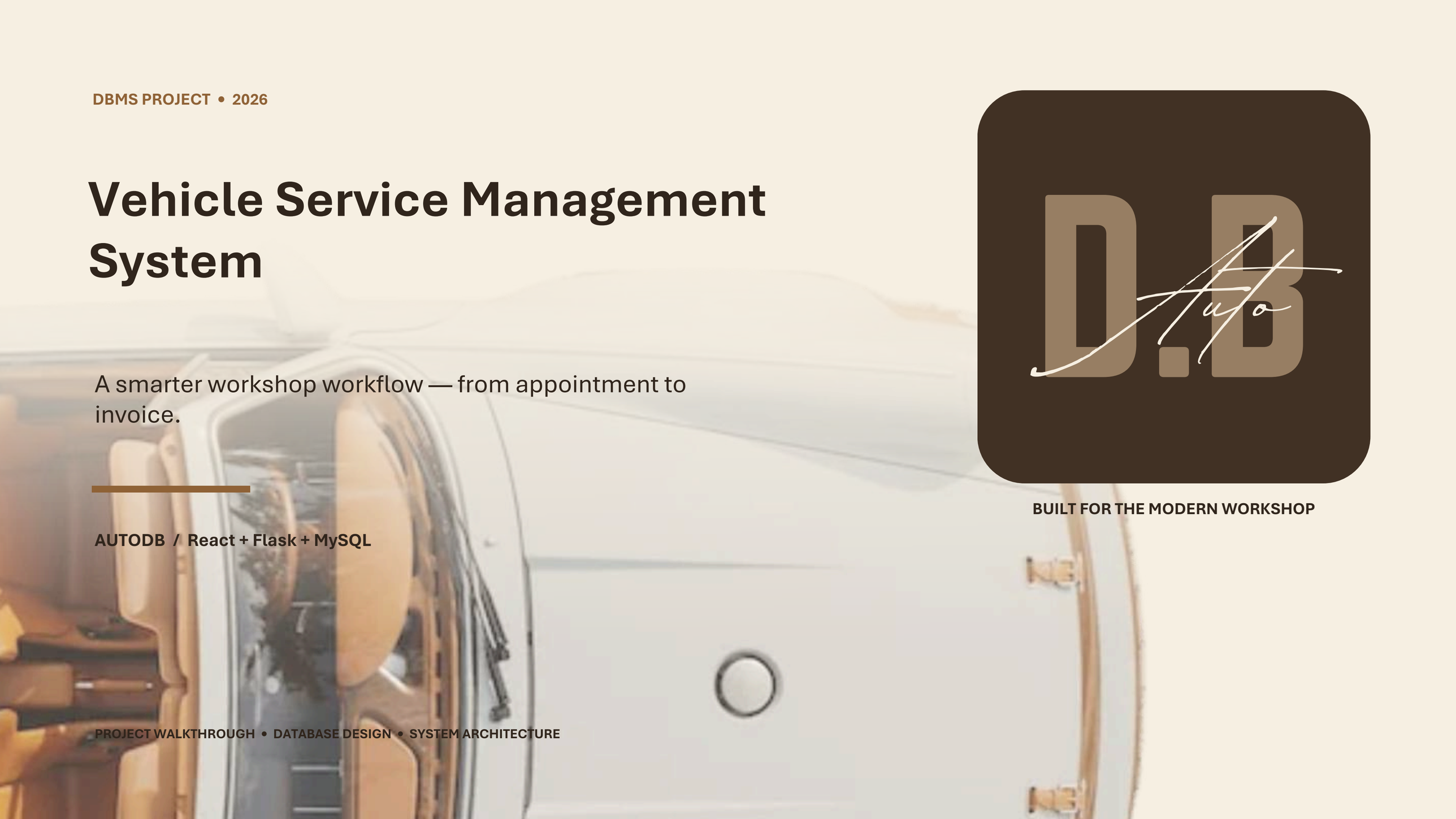
DBMS PROJECT • 2026

Vehicle Service Management System

A smarter workshop workflow — from appointment to invoice.

AUTODB / React + Flask + MySQL

PROJECT WALKTHROUGH • DATABASE DESIGN • SYSTEM ARCHITECTURE



BUILT FOR THE MODERN WORKSHOP

One workshop. One connected system.

The goal is to turn a multi-step service journey into a clear, trackable digital workflow.

Before

Scattered records, repeated manual updates, and limited visibility into service progress.

With AutoDB

Customer, vehicle, appointment, mechanic, parts, and billing data are connected.

Outcome

A more consistent process, traceable records, and clearer decisions at every step.

CORE IDEA Capture the data once — use it throughout the service lifecycle.

The workshop lifecycle, end to end

The application connects operational tasks that are often handled separately.

01 **Customer & vehicle**

Profiles, vehicle details, service history

02 **Appointments**

Booking, review, mechanic assignment

03 **Inspection & estimate**

Findings, labour, proposed parts and cost

04 **Approval & service**

Customer decision gates the work

05 **Inventory**

Parts usage, stock checks and reorder levels

06 **Invoice & payment**

Final charges, invoice records and reporting

A polished interface for a practical workflow

Drop in your real homepage/dashboard screenshot to make this slide feel like a product demo.



Visual direction

Midnight navy, mint accents, spacious cards, and a premium automotive feel.

Designed around tasks

Clear navigation and role-specific areas help users focus on what they need to do.

Demo tip

Use a full-width screenshot with browser chrome cropped out for a cleaner look.

Different roles. Different responsibilities.

Role checks are enforced in the backend, not only by hiding or showing frontend pages.

Customer

• Manage vehicles • Book appointments • Review inspection estimates • Approve or reject estimates • View invoices and service history

Mechanic

• View assigned services • Record inspection findings • Propose labour and parts • Record actual work and parts • Complete approved services

Admin / workshop staff

• Review appointments • Assign mechanics • Manage parts and stock • View dashboard and revenue report • Access billing operations

Authentication: JWT access tokens • Passwords: hashed • Protected routes: role-checked

From booking to a completed service

A guided sequence helps protect the integrity of service and billing records.

01
Book
Customer adds vehicle + complaint

→

02
Inspect
Mechanic records findings

→

03
Estimate
Labour and parts are proposed

→

04
Approve
Customer accepts or rejects

→

05
Perform
Actual work and parts are recorded

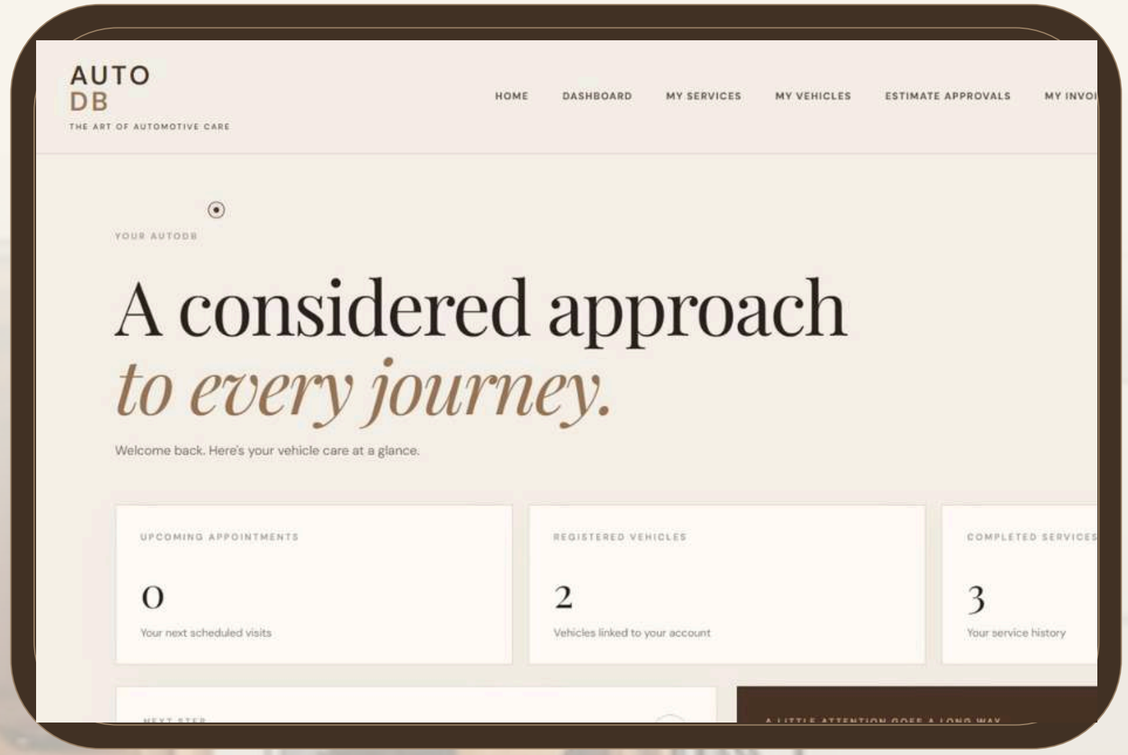
→

06
Complete
Stock and invoice are updated

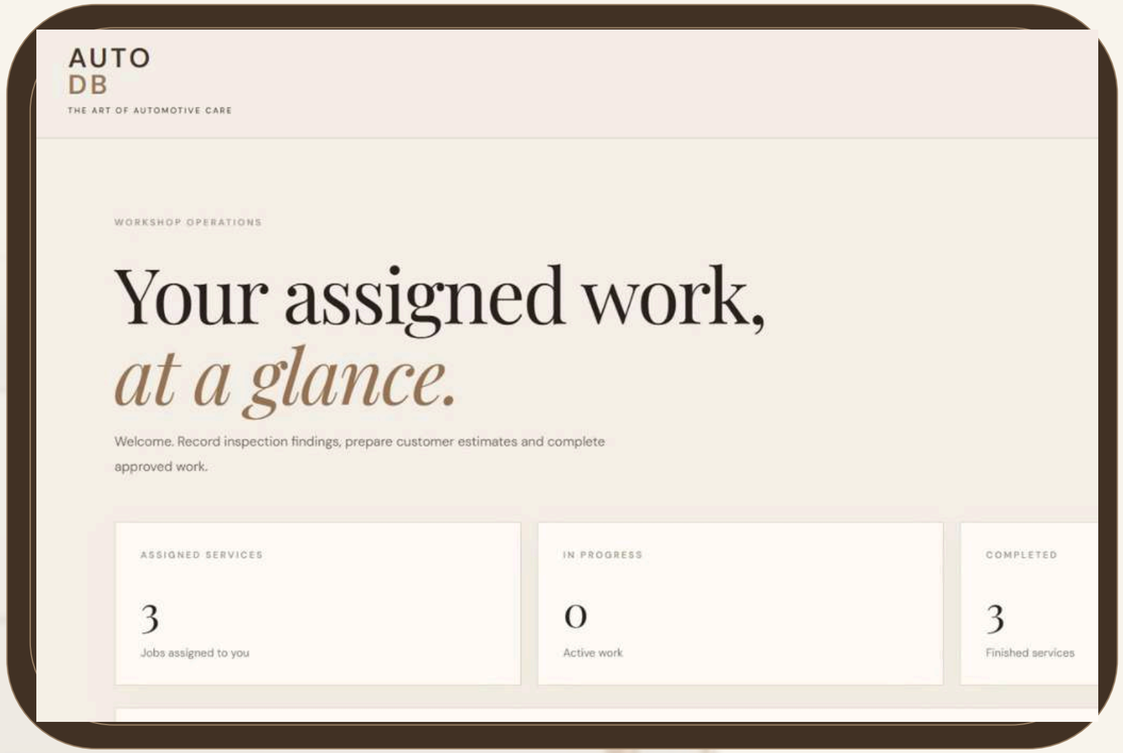
IMPORTANT RULE A service cannot start until its estimate has been approved.

Two views of the same service journey

Add screenshots of the customer dashboard and mechanic service/estimate screen.



CUSTOMER • Visibility and approval



MECHANIC • Inspection and execution

Three layers, one connected application

The React client calls a Flask API, which reads and writes structured records in MySQL.

FRONTEND

React + Vite JavaScript / CSS

→

BACKEND API

Python + Flask Auth + business rules

→

DATABASE

MySQL Relational tables + keys

API adapter (api.js) • JWT Authorization header • mysql-connector-python

Relational data models the real workshop

The schema separates core entities while connecting them through keys and service records.

Customer

Customer identity and contact details

Vehicle

Vehicle details linked to an owner

Appointment

Requested service and scheduling

ServiceRecord

Service status and execution record

ServiceTask

Work items linked to a service

Part / PartUsage

Stock catalogue and consumed parts

Invoice / Payment

Charges and payment history

Employee / UserAccount

Staff identity, login and role

Design principles: primary keys • foreign keys • constraints • normalized entities • transaction-aware updates

SQL does more than store data

The database layer supports creation, retrieval, updates, relationships, and consistency.

DDL

CREATE TABLE, ALTER TABLE Define schema and evolve it with migrations.

DML

INSERT, UPDATE, DELETE Maintain customer, service, parts and invoice records.

Queries

SELECT, WHERE, JOIN Retrieve dashboards, history, billing and inventory views.

Relationships

Foreign keys connect related records and reduce orphan data.

Transactions

Completion logic groups inventory and invoice changes into a unit of work.

Verification

Audit queries inspect stock, service status and invoice outcomes.

Guardrails keep the workflow trustworthy

Important decisions are checked by the backend before data changes are accepted.

Approval gate	A service can start only after the customer approves the estimate.
Actual usage	Actual parts consumed are recorded and checked against the estimate.
No double deduction	A StockDeducted flag helps prevent the same usage row reducing stock twice.
Atomic completion	Service completion groups inventory and invoice updates in a database transaction.

What has been checked — and what still needs proof

A strong presentation should be transparent about the difference between build checks and end-to-end validation.

Automated checks in the repo

• Frontend dependency install, lint and production build • Backend Python syntax compilation • GitHub Actions workflow for repeatable validation

Still worth validating manually

• MySQL setup and migrations on a clean database • Complete customer → mechanic → invoice journey • Payment endpoints with the Payment table present • Automated database/API integration tests

Takeaway Build checks are useful, but they do not replace a full workflow test against MySQL.

Show the story, not just the screens

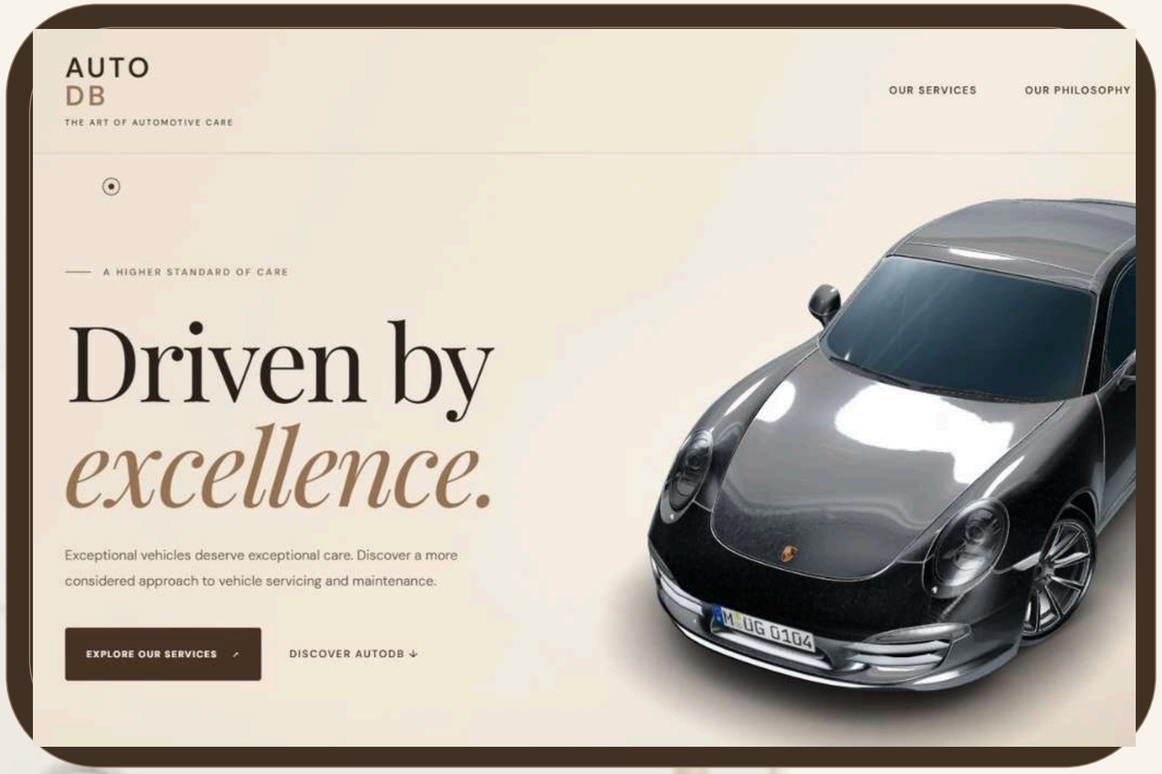
Suggested screenshot sequence for a 60–90 second walkthrough.

01 Landing / login
Introduce the visual style and entry points.

02 Customer booking
Show vehicle details and appointment creation.

03 Estimate approval
Explain the approval gate and customer decision.

04 Service completion
Show actual parts, stock impact, and invoice result.



Thank you.

A connected database. A guided workflow. A clearer way to run a workshop.

Explore the source code

github.com/coding-bee7/VehicleServiceDB

